Smart Cinema System

Software Requirements Specification

Version 1.0

9/22/2026

Group #13
Khang Do
Minh Nguyen
Phu Pham

Prepared for
CS 250- Introduction to Software Systems
Instructor: Gus Hanna, Ph.D.
Fall 2026

Revision History

Date	Description	Author	Comments
09/22/2026	Version 1.0	Phu Pham	
09/22/2026	Version 1.0	Minh Nguyen	
09/22/2026	Version 1.0	Khang Do	

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	Dr. Gus Hanna	Instructor, CS 250	

Table of Contents

Revision History **2**
Document Approval **2**
1. Introduction **1**
1.1 Purpose 1
1.2 Scope 1
1.3 Definitions, Acronyms, and Abbreviations 1
1.4 References 1
1.5 Overview 2
2. General Description **2**
2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Characteristics 2
2.4 General Constraints 2
2.5 Assumptions and Dependencies 2
3. Specific Requirements **3**
3.1 External Interface Requirements 3
3.1.1 User Interfaces 3
3.1.2 Hardware Interfaces 3
3.1.3 Software Interfaces 3
3.1.4 Communications Interfaces 3
3.2 Functional Requirements 3
3.2.1 Account and role management 3
3.2.2 Movie browsing and showtime search 4
3.3 Use Cases 4
3.3.1 Use case #1 4
3.3.2 Use case #2 4
3.3.3 Use case #3 4
3.3.4 Comprehensive UML Use-Case Diagram 5
3.4 Classes / Objects 5
3.4.1 Account 5
3.4.2 Movie 5
3.4.3 Auditorium and Seat 5
3.4.4 Showtime 5
3.5 Non-Functional Requirements 6
3.5.1 Performance 6
3.5.2 Reliability 6
3.5.3 Availability 6
3.5.4 Security 6
3.5.5 Maintainability 6

3.5.6 Portability..... 7
3.6 Inverse Requirements.....7
3.7 Design Constraints..... 7
3.8 Logical Database Requirements.....7
3.9 Other Requirements..... 7
4. Analysis Models.....7
4.1 Sequence Diagrams..... 7
4.2 Data Flow Diagrams (DFD)..... 8
4.3 State-Transition Diagrams (STD).....9
5. Change Management Process.....10
A. Appendices.....10
A.1 Appendix 1..... 10
A.2 Appendix 2..... 10
6 Software Design Specification.....10
6.1 System Description.....10
6.2 Software Architecture..... 10
6.3 UML Class Diagram.....11
6.4 Shared Data Types.....13
6.5 Main Design Rules.....13
6.6 Classes, Attributes and Operations..... 13
Class Descriptions..... 14
6.7 Application Service Interfaces..... 15
6.8 Development Plan and Timeline.....17
7. Test Plan.....18
7.1 Design Specification Update.....18
7.2 Test Approach and Test Sets.....18
Sample data.....19
7.3 Test Cases and Execution..... 19
Running and recording tests.....19

1. Introduction

The purpose of this Software Requirements Specification is to define the Cinema Systems' scope, user roles, functional/non-functional requirements, interfaces, use cases and analysis models. The Cinema System provides customer access to searching for movies and their showtimes; selecting seats; purchasing tickets; and viewing their booking history. Additionally, employees have the ability to sell/verify tickets; manage movie scheduling; cancel purchases; and generate reports.

1.1 Purpose

This SRS is a definition of the cinema system requirements; it will serve as a reference for Cinema System Developers, Testers, Cinema Managers and Instructors to link system characteristics, use cases, models and tests using the requirement ID.

1.2 Scope

The Cinema System supports one cinema with multiple auditoriums. Customers can view showtimes, select seats, pay, receive QR tickets, and cancel bookings when allowed. Staff can sell and verify tickets, manage schedules, and handle sales and refunds. Version 1.0 excludes concessions, loyalty programs, streaming, and multi-cinema management.

1.3 Definitions, Acronyms, and Abbreviations

Term	Meaning
Showtime	One screening of one movie in one auditorium at a scheduled time.
Hold and booking	A hold reserves seats temporarily. A booking records seats, contact, price, and lifecycle status.
QR ticket	A ticket containing an unpredictable token encoded as a QR code.
Idempotent operation	Repeating a request or callback does not repeat its business effect.
Reconciliation	Checking provider records to resolve an uncertain payment or refund.

1.4 References

William Y. Arms, Scenarios and Use Cases, Cornell University CS 5150. Supplied as lecture-use-cases.pdf; publication date not stated. Used for actors, scenarios, and use case descriptions.

E-Store Project Software Requirements Specification, Version 4.0, Group-1, April 16, 2007. Supplied as SRS4.0.doc. Used as an example of detailed functional requirements.

Shock Force Software Team, Software Requirements Specification, October 20, 2009. Supplied as SoftwareRequirementsTemplate.docx. Used as an example of operational scenarios and analysis models.

1.5 Overview

Chapter 2 describes the product, users, assumptions, and business rules. Chapter 3 defines detailed requirements and use cases. Chapter 4 presents analysis models. Chapter 5 controls requirement changes. Appendix A maps requirements to acceptance tests; Appendix A.2 provides sample data and an open-decision register.

2. General Description

2.1 Product Perspective

The Cinema System is a browser-based application that has an inventory of seats which are available for both customer use and staff use. The Cinema System connects to external payment processing systems as well as email systems; however, the projection equipment is separate from the Cinema System.

2.2 Product Functions

The system supports customer booking and cancellation, box-office sales and payments, ticket admission, and management of movies, seats, showtimes, prices, staff roles, and operations.

2.3 User Characteristics

The system includes Customers, Cashiers, Ushers, and Managers with different access levels, plus external payment and email services for transactions and notifications.

2.4 General Constraints

This system will need an Internet connection in order to access the server for managing time, price, seat availability and ticket sales. This system does not use any particular programming language or type of database.

2.5 Assumptions and Dependencies

Version 1.0 supports one cinema using USD and local venue time. Bookings contain 1–8 seats for one showtime, with 5-minute holds and no partial cancellation or seat exchange. Sales stop at show start, while cancellations are allowed at least 30 minutes before the show if tickets are unused. Managers can cancel future shows and issue refunds. Admission is valid from 30 minutes before the show until the movie ends, with one entry per ticket. Pricing includes seat price, fees, and tax, and showtimes must include at least 20 minutes of turnaround. Card and

cash refunds must be properly recorded, and payment or email failures must not create invalid bookings.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The system should be simple to use. Customers can find showtimes, review orders, buy tickets, receive confirmation, and view booking history, while staff only access features needed for their roles.

3.1.2 Hardware Interfaces

The system will operate in common web browsers across all computer types, tablet devices, and mobile phone devices. Tickets are able to be scanned with the use of a device's camera, or manually entered into the system. Box-office tickets may be printed from a standard printer.

3.1.3 Software Interfaces

The system securely integrates with payment, e-mail, and data services; it also prohibits duplicate payment transactions, duplicate ticket transactions, and duplicate seat transactions.

3.1.4 Communications Interfaces

The system utilizes HTTPS to verify event providers, prevents duplicate actions from occurring, monitors the status of external services utilized by the system, and requires on-line ticket verification.

3.2 Functional Requirements

Each feature below follows the course template: introduction, inputs, processing, outputs, and error handling. Detailed processing statements carry the IDs. All externally supplied data must be validated before it changes a record.

3.2.1 Account and role management

3.2.1.1 Introduction

Customer identity and staff access are managed separately.

3.2.1.2 Inputs

Email, password, recovery token, staff account, and assigned role.

3.2.1.3 Processing

Customers will be able to browse all of the movie releases they want to see, find and sort through the available showtimes, and view important showtime information (such as time, auditorium, price, and how many seats are left) without being able to choose sold-out, canceled, unpublished, or already-started showtimes.

3.2.1.4 Outputs
An authenticated session, verified customer profile, or audited staff-role change.

3.2.1.5 Error Handling
Reject duplicate account emails and expired tokens. Return a generic response to a recovery request so it does not reveal whether an account exists.

3.2.2 Movie browsing and showtime search

3.2.2.1 Introduction

Customers find screenings that can still be booked.

3.2.2.2 Inputs

Movie title, date, language, format, and optional filters.

3.2.2.3 Processing

The system shows available seats, holds selected seats for up to five minutes, releases expired holds, and rechecks availability before purchase.

3.2.2.4 Outputs

Matching movies and showtimes with current availability.

3.2.2.5 Error Handling

Invalid dates produce a validation message. A stale listing never overrides the final inventory check.

3.3 Use Cases

3.3.1 Use case #1

Customers are able to establish, check validity of, log-in to, and modify their accounts. Managers have the ability to assign/remove employee roles with all modifications being recorded and implemented at the time of modification.

3.3.2 Use case #2

Customers are able to search through movie listings; sort by showtime; see pricing and availability; and access a seating chart for any available showing.

3.3.3 Use case #3

The Customer or Cashier selects 1–8 seats for an available showtime. The system checks all selected seats at once and either holds all of them or none. A successful hold lasts 5 minutes and includes a price snapshot and hold reference. If any seat becomes unavailable or the hold expires, the reservation fails or is released.

3.3.4 Comprehensive UML Use-Case Diagram

The diagram summarizes the major functions and external interactions for Customers, Cashiers, Ushers, Managers, the payment provider, and the email service.

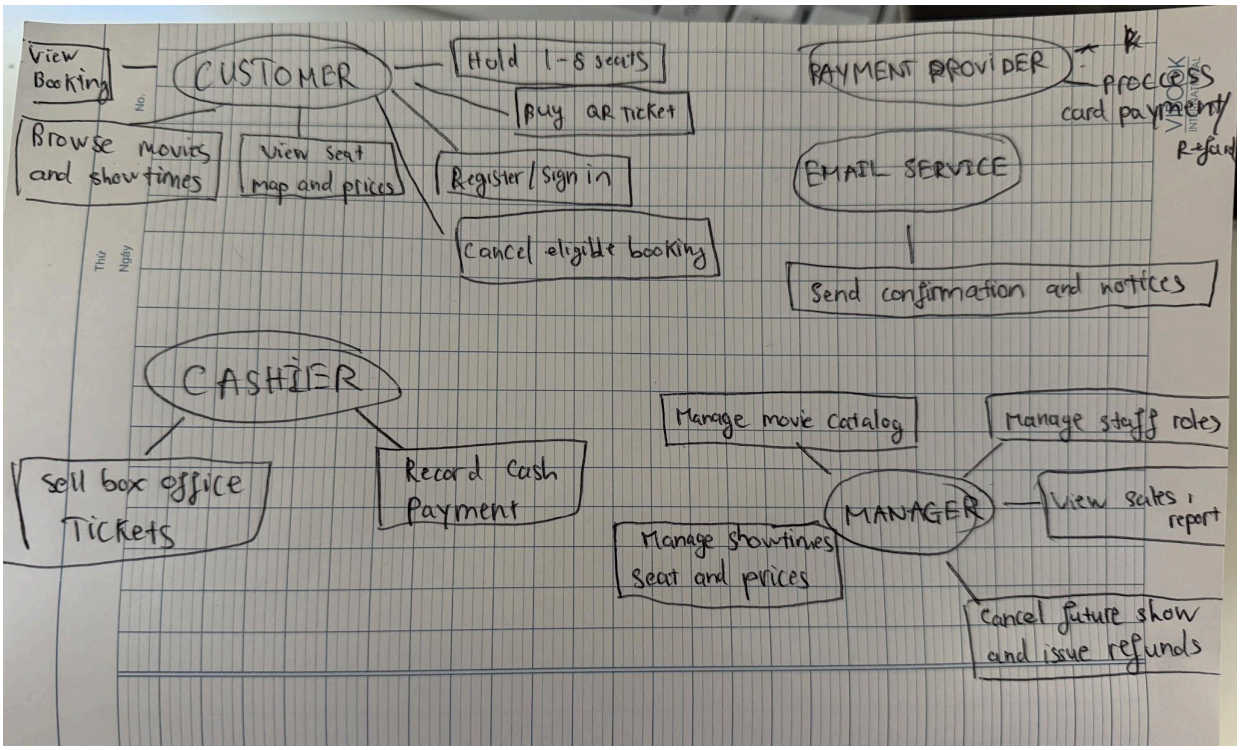


Figure UC-1. UML use-case diagram for Smart Cinema System.

3.4 Classes / Objects

3.4.1 Account

Stores the account information of users (name, email etc.), authentication data (password hash), user profile information (date of birth), role assignment to each user (admin, editor), and account status.

3.4.2 Movie

Stores all details related to movies; e.g. - title, length/runtime, rating (IMDB), language, format (DVD, Bluray), archival status.

3.4.3 Auditorium and Seat

Stores auditorium layouts and seat plans for all auditoriums. It also contains seat label information for each seat within an auditorium and accessibility information.

3.4.4 Showtime

Stores the schedule of showtimes in auditoriums with associated pricing, start times, and current status of the showtime.

3.5 Non-Functional Requirements

The following are proposed measurable acceptance targets for team/instructor review; they are not measured implementation results. Test with 500 simultaneous browsing sessions, 50 checkout sessions, up to 10 auditoriums of 300 seats each, and client round-trip latency at or below 100 ms. Measure third-party payment and email delays separately.

3.5.1 Performance

Under the prescribed test load, at least 95 percent of requests for each operation, including movie searches, showtimes, seat maps, seat holds, and ticket validation, must be completed within 2 seconds after the server receives them (excluding third-party processing time). At least 95 percent of internal payment confirmations must be completed within 2 seconds after receiving a verified response from the payment provider. When a five-minute seat hold expires, the seats must become available again within 30 seconds. Concurrent testing must also show that the same seat cannot be assigned to more than one active booking for the same showtime.

3.5.2 Reliability

The system must prevent duplicate seat sales, payment charges, and ticket admissions in 1,000 concurrent test cases. Confirmed booking, payment, and refund records must remain unchanged after a simulated restart. Backup testing must show an RPO of 15 minutes or less and an RTO of 4 hours or less. Unknown payment outcomes must be rechecked within five minutes after the provider's services become available again.

3.5.3 Availability

The Catalog and Booking Service must maintain at least 99.5% monthly availability, excluding scheduled downtime, based on external health checks. If the Email Delivery System is unavailable, paid bookings and QR tickets must remain accessible, and queued emails must be resent within 5 minutes after service returns. If the Payment Provider is unavailable, users can still browse, but card payments must show a "pending" or "error" message instead of an unverified successful transaction.

3.5.4 Security

All staff-only actions must use server-side role checks, and unauthorized requests must be rejected. Passwords must use salted adaptive hashing, never plaintext. Accounts must be locked for 15 minutes after five consecutive failed login attempts. Inactive sessions must end after 30 minutes, and password reset tokens must expire within 30 minutes. All payment callbacks must pass provider signature verification, and guests cannot assign another customer's account ID to a booking.

3.5.5 Maintainability

All reservation, payment, refund, ticket, and staff-role changes must create an audit event containing the Request ID, Actor (if available), Action, and Timestamp. Changes to critical Business Rules must update the requirements, design documents, and regression tests before merging. Before release, all critical tests must pass, including seat holds, payment confirmation, cancellation/refund, and one-time ticket admission. Timeout and pricing policies should also be configurable without changing the underlying Business Logic.

3.5.6 Portability

Customer and staff interfaces must support the two latest stable versions of Chrome, Edge, Firefox, and Safari, along with current mobile Safari and Chrome. All main functions, including browsing, seat selection, payments, and ticket validation, must work on screen widths from 360 px to 1440 px without horizontal scrolling. Critical functions must support keyboard navigation, clear text labels, and status indicators that do not rely only on color. Tickets must allow manual entry as an alternative to QR scanning, and box-office tickets must use standard browser printing without special hardware or software.

3.6 Inverse Requirements

The system will not validate an unpaid online reservation or a credit card reservation based on success messages from the browser. The system will not sell the same seat at the same show time to two different active reservations.

3.7 Design Constraints

Sales on line and in the ticket office shall be carried out via one authoritative system for allocation and data storage supporting transactions and unique entries. Client-side validations are optional. Money amounts in the database shall be stored in integers denoting cents, and time stamps in UTC instants. Dates displayed or reported shall be presented using the venue time zone.

3.8 Logical Database Requirements

The system must ensure all events, request, transaction, refund, and ticket identifiers are unique. Each successful payment can generate at most one refund. Tickets are retrieved using token hashes, with encrypted token backups for reprinting or delivery. Booking records must preserve the original movie, showtime, seat, payment, and billing details so later catalog changes do not alter past receipts.

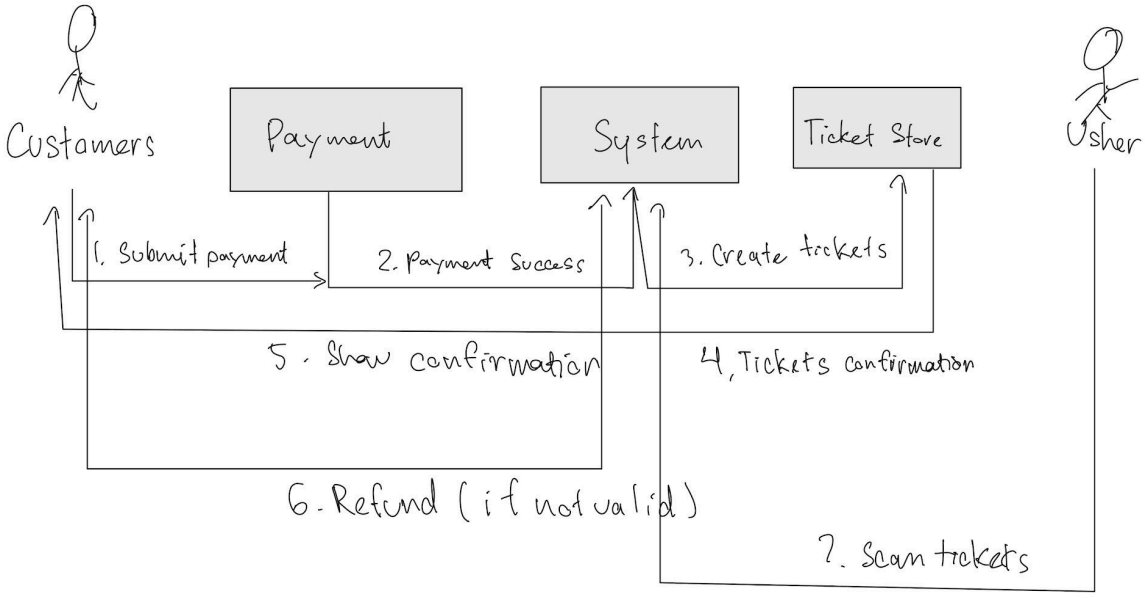
3.9 Other Requirements

The customer confirmation and cancellation screens must show the cutoff for cancellation using the time zone of the venue and the refund status in clear language. Staff support must provide explanations for the following cases: expired hold, uncertain payment, double scan, printing error, and cash refund on hold.

4. Analysis Models

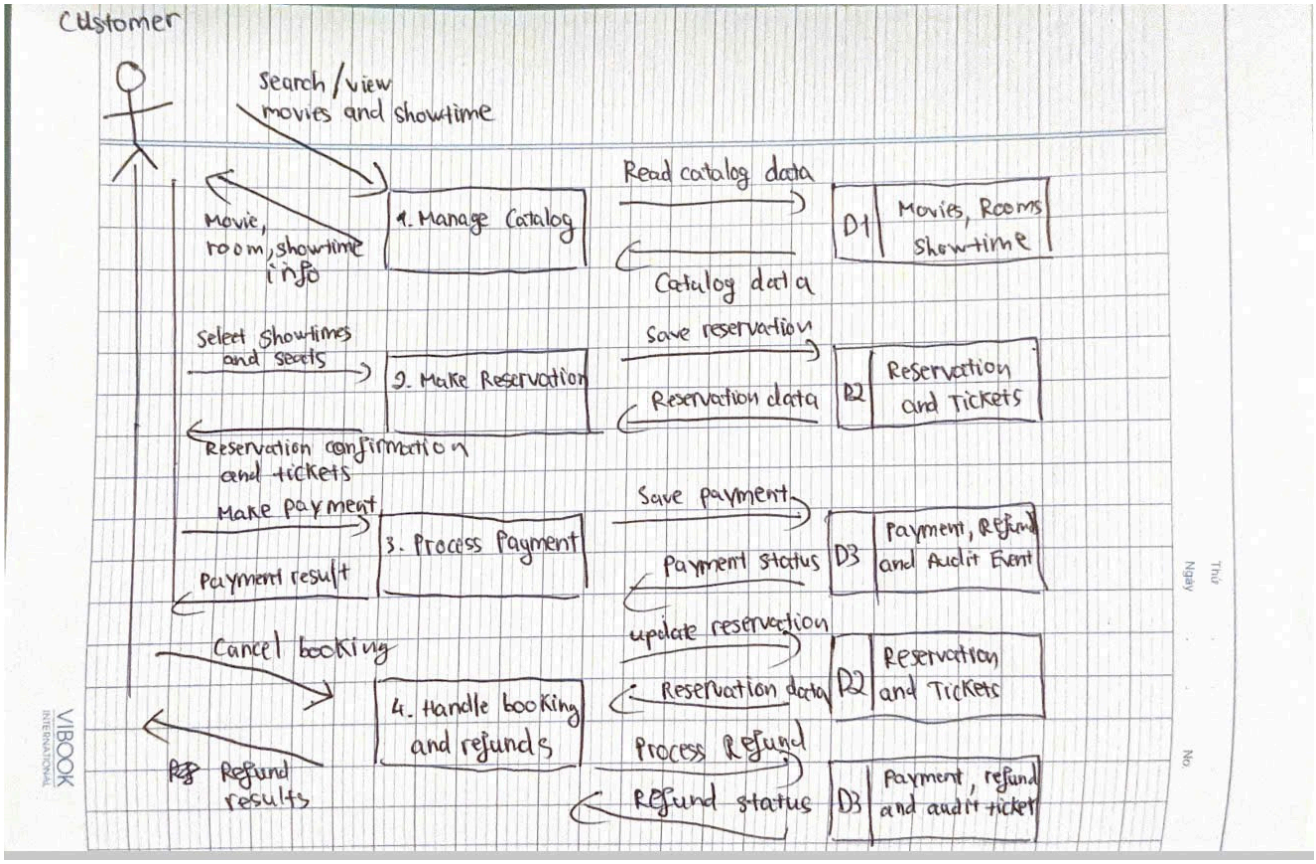
4.1 Sequence Diagrams

Payments confirm bookings only when seats and showtimes remain valid; late payments are refunded. Tickets can be admitted only once, while duplicate scans are rejected and logged.



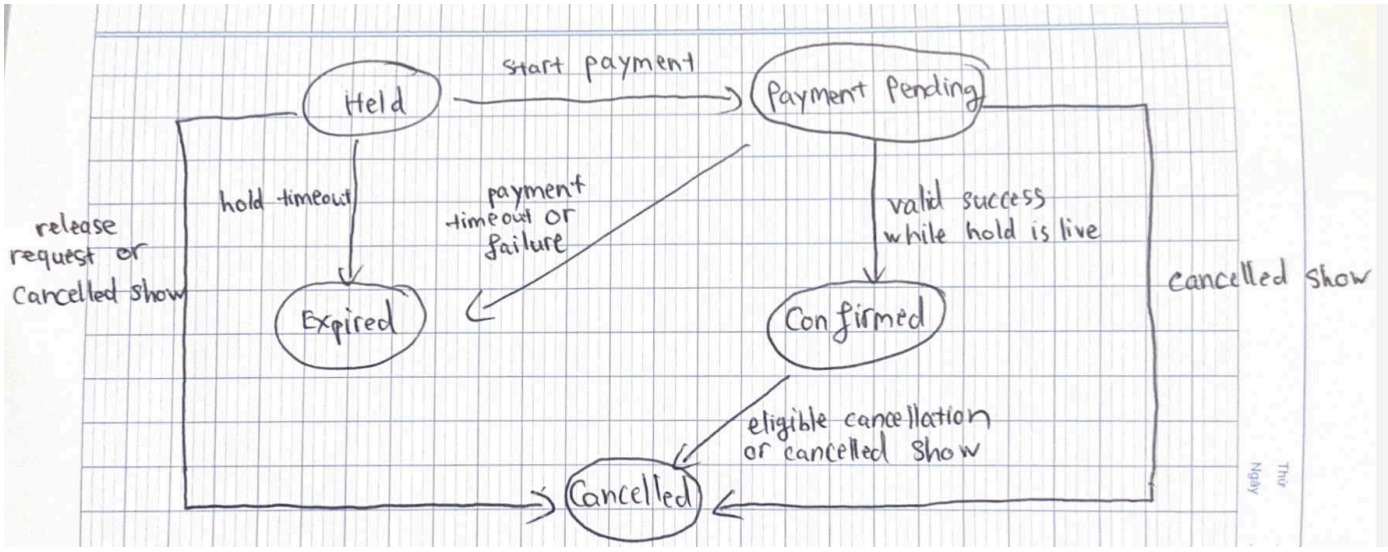
4.2 Data Flow Diagrams (DFD)

Data Flow Diagram is divided into two diagrams: customers and employees' perspectives on the four processes. The database D1 contains information about movies, rooms, and show times; D2 contains information about reservations and tickets; D3 contains information about payments, refunds, and audit information.



4.3 State-Transition Diagrams (STD)

Booking State Life Cycle starts at Seat Hold stage. From there, Booking moves on to Payment Pending if it's a card transaction or to Confirmed if it’s cash. Booking cannot go past the Expired or Cancelled stages. The Refund state is tracked via a different Refund object.



5. Change Management Process

Change requests may be submitted by a customer representative, a developer, tester or even the instructor. Change request provides for recording the change ID, reason, related requirements, suggested text changes and expected benefit. Verbal change requests must be written down prior to changing baseline. The author analyzes impact on business policies, use cases, data, diagrams, acceptance tests and scope. Migration effort related to existing holds, bookings, tickets and payment is identified.

A. Appendices

A.1 Appendix 1

With regards to testing at a fundamental level, the system must undergo tests to determine if the users are able to self-register with the system; log-in to the system; search for movies and show times; reserve seating; and cancel their reservations.

A.2 Appendix 2

This appendix has sample data for use in testing the system. Sample data includes Smart Cinema, Auditorium 1, a sample film showing at 7:00pm, a five-minute seat hold, booking from one to eight seats and QR code tickets. These are examples to test the system and will not create additional system requirements.

6 Software Design Specification

6.1 System Description

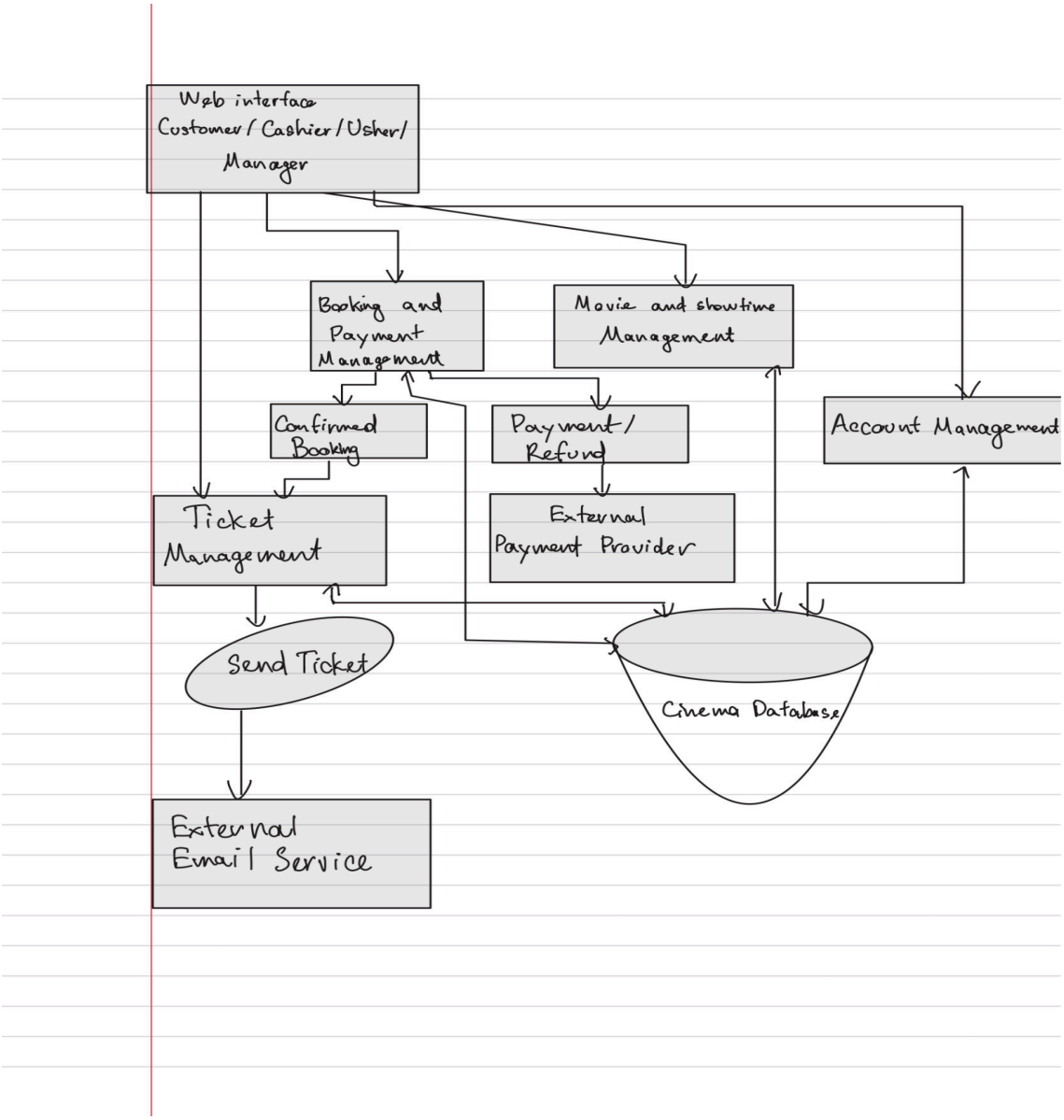
Smart Cinema System is a Web-Based Application which enables customers to browse movie listings, choose seats, purchase tickets, and see their past ticket purchases. Customers may also cancel their tickets if given permission. Cashiers can process ticket sales. Ushers can verify tickets for the correct showtime and seat location. Managers can create/delete movies, schedule showtimes, set pricing, and manage employee login credentials.
The system uses one shared seat inventory to prevent double booking. Customers can select 1–8 seats, which are held for 5 minutes before payment. After a successful payment, the system generates QR tickets. Each ticket can only be used once. Customers can cancel their bookings at least 30 minutes before showtime if the tickets have not been used.

6.2 Software Architecture

The Smart Cinema System has a web-based user interface that interacts with four major parts:
- Account Management controls user account creation, login, and staff authorization.
- Movie & Showtime Management controls movie availability, showtimes, theater room assignments, and seat availability.
- Booking & Payment Management allows users to reserve seats, make payments, cancel reservations, and receive refunds.

- Ticket Management creates QR-coded tickets and verifies them when customers enter the theater.

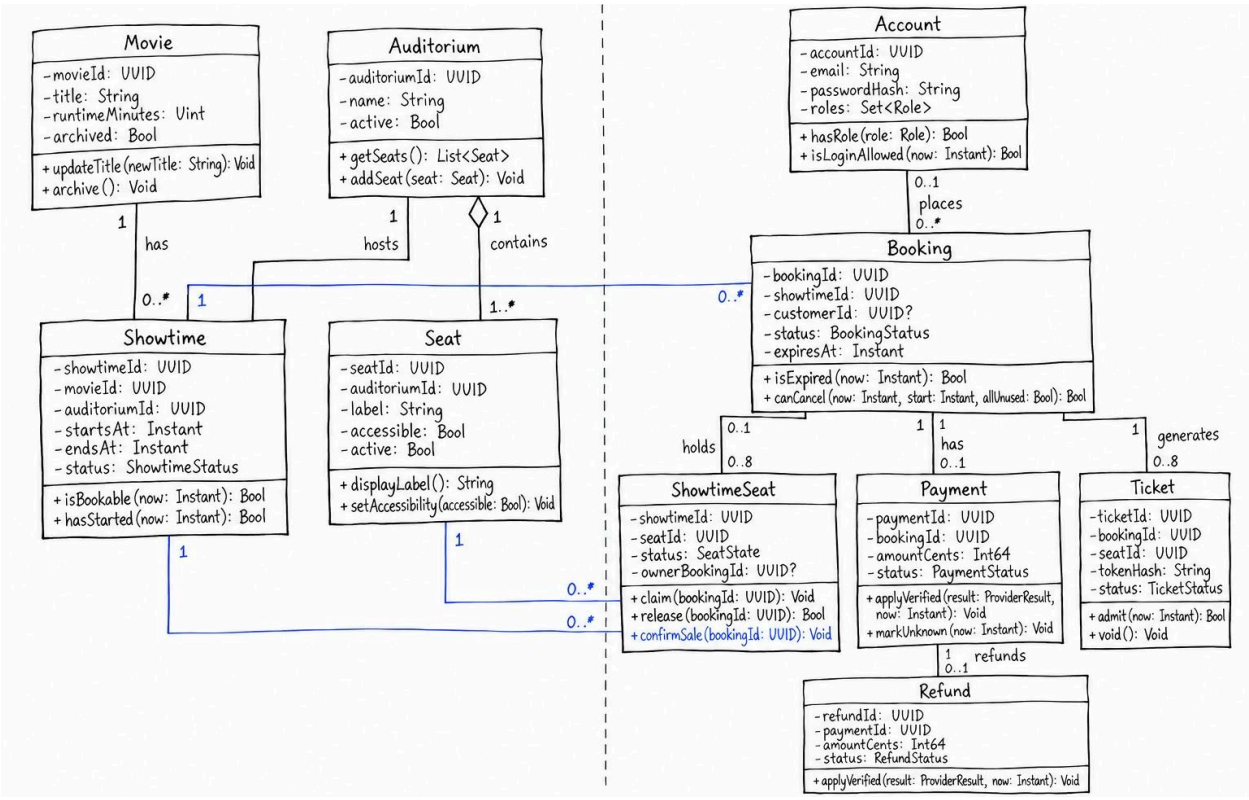
Each part accesses a single database to create, update, and delete information. Booking & Payment Management interacts with an external payment service to handle transactions and refunds. After successful payment, Ticket Management creates tickets and sends them to customers using an external email service.



6.3 UML Class Diagram

The UML Class Diagrams are representative of all the significant classes in the Smart Cinema System. All of these classes have properties (attributes), types of data that each property can hold (data types), and functionality (operations). The lines drawn between the classes represent how they relate to one another. The first diagram illustrates connections between movies, show

times, auditoriums, and seat positions. The second diagram illustrates connections between account related functions like booking, processing payments, handling refunds and producing tickets.



6.4 Shared Data Types

Several data types exist in this system to represent information. UUID is utilized to provide a unique identifier for each item. String is utilized to display text based data. Bool is utilized to represent either a true or false value. UInt is utilized to represent any non-negative number. Money in cents is represented using Int64. Instant is utilized to represent date and time based data in Universal Coordinated Time (UTC). List<T> and Set<T> are utilized to represent collections of items. T? is utilized to represent an optional value. Status types are utilized in this system to monitor the status of various aspects of operations including BookingStatus (status of bookings), PaymentStatus (status of payments), RefundStatus (status of refunds) and TicketStatus (status of tickets).

6.5 Main Design Rules

The system uses one shared seat record for each seat in a showtime. Customers can hold 1–8 seats for 5 minutes, and the system must reserve all selected seats or none. A booking is confirmed only when the hold is still valid, the seats still belong to that booking, and payment is verified. If a payment is successful even after the hold expires then the system should give back the money rather than confirming the reservation. The control over who has access to staff members is based on their assigned role; passwords and sessions must be secured. Each refund will have its own tracking number (similar to booking IDs), and each QR code on your ticket will only work for one use.

6.6 Classes, Attributes and Operations

Class / purpose	Important attributes	Major operations
Account — Stores user information and staff roles	accountId: UUID email: String passwordHash: String roles: Set<Role>	hasRole(role: Role): Bool isLoginAllowed(now: Instant): Bool
Movie — Stores movie information	movieId: UUID title: String runtimeMinutes: UInt archived: Bool	updateTitle(newTitle: String): Void archive(): Void
Auditorium — Stores theater room information and its seats	auditoriumId: UUID name: String active: Bool	getSeats(): List<Seat> addSeat(seat: Seat): Void
Seat Seat position and accessibility	seatId: UUID auditoriumId: UUID label: String accessible: Bool active: Bool	displayLabel(): String setAccessibility(accessible: Bool): Void

Class / purpose	Important attributes	Major operations
Showtime — Stores movie schedules and screening times.	showtimeId: UUID movieId: UUID auditoriumId: UUID startsAt: Instant endsAt: Instant status: ShowtimeStatus	isBookable(now: Instant): Bool hasStarted(now: Instant): Bool
ShowtimeSeat — Tracks seat availability for each showtime.	showtimeId: UUID seatId: UUID status: SeatState ownerBookingId: UUID?	claim(bookingId: UUID): Void release(bookingId: UUID): Bool
Booking — Manages seat reservations and cancellations.	bookingId: UUID showtimeId: UUID customerId: UUID? status: BookingStatus expiresAt: Instant quote: PriceQuote	isExpired(now: Instant): Bool canCancel(now: Instant, start: Instant, allUnused: Bool): Bool
Payment — Stores payment information and status.	paymentId: UUID bookingId: UUID method: PaymentMethod amountCents: Int64 status: PaymentStatus idempotencyKey: UUID	applyVerified(result: ProviderResult, now: Instant): Void markUnknown(now: Instant): Void
Refund — Tracks refunds for payments.	refundId: UUID paymentId: UUID amountCents: Int64 status: RefundStatus	applyVerified(result: ProviderResult, now: Instant): Void
Ticket — Stores QR tickets and admission status.	ticketId: UUID bookingId: UUID seatId: UUID tokenHash: String status: TicketStatus	admit(now: Instant): Bool void(): Void

Class Descriptions

The Account: Stores user information, including email, password hash, and staff roles. The hasRole() method checks the user's permission, while isLoginAllowed() checks whether the account can log in.

The Movie: Stores the movie title, runtime, and archive status. The updateTitle() method changes the title, while archive() removes the movie from active listings.

Showtime: Stores the movie, auditorium, and start and end times. The methods check whether a showtime can still be booked or has already started.

ShowtimeSeat: Tracks whether a seat is available or held for a specific showtime. The methods allow the system to reserve or release the seat.

Booking: Stores reservation details, including the customer, showtime, expiration time, and price information. The methods check whether a reservation has expired or can be canceled.

Payment: Stores the payment amount, method, and status. The methods update verified payments and handle payments with unknown results.

Refund: Stores refund amounts and their current status. Its method updates the refund after verification from the payment provider.

Ticket: Stores ticket information and a protected QR token hash. The methods allow the system to admit a customer or invalidate a ticket.

Auditorium: Stores theater room information and its seats. The getSeats() method returns the seats in the auditorium, while addSeat() adds a new seat.

Seat: Stores seat information, including its label and accessibility status. The displayLabel() method returns the seat label, while setAccessibility() updates whether the seat is accessible.

6.7 Application Service Interfaces

The Smart Cinema System uses six application services to handle its main functions. These services work within the four main components described in Section 6.2. Each service checks user input and verifies permissions when needed.

Service	Typed operation	Responsibility
AuthService	register(email: String, password: String): Account login(email: String, password: String): Bool authorize(sessionToken: String, role: Role): Bool	Manages user accounts, login, and staff permissions.

Service	Typed operation	Responsibility
CatalogService	search(keyword: String): List<Showtime> seatMap(showtimeId: UUID): List<ShowtimeSeat> saveShowtime(showtime: Showtime): Void	Displays movies, showtimes, and available seats. Allows managers to update showtimes.
BookingService	createHold(customerId: UUID?, showtimeId: UUID, seatIds: List<UUID>): Booking expireHolds(now: Instant): Void history(customerId: UUID): List<Booking> cancelBooking(bookingId: UUID, now: Instant): Bool	Holds 1–8 seats for 5 minutes, releases expired holds, and manages booking history and cancellations.
PaymentService	startCard(bookingId: UUID): Payment recordCash(bookingId: UUID, amountCents: Int64): Payment handleProviderEvent(result: ProviderResult, now: Instant): Void processRefund(paymentId: UUID): Refund	Processes card and cash payments, verifies payment results, and handles refunds.
TicketService	issue(bookingId: UUID): List<Ticket> deliver(actor: ActorContext, bookingId: UUID, mode: DeliveryMode, key: UUID): Result<TicketBundle> validate(actor: ActorContext, qrToken: String, attemptId: UUID): Result<AdmissionDecision>	Creates QR tickets after confirmed payment, delivers tickets, and prevents tickets from being used more than once.

Service	Typed operation	Responsibility
OperationsService	cancelShow(actor: ActorContext, showtimeId: UUID, reason: String, key: UUID): Result<CancellationSummary> salesReport(actor: ActorContext, range: LocalDateRange): Result<SalesReport> sendPendingNotifications(now: Instant, limit: UInt): UInt	Allows managers to cancel shows, generates sales reports, and retries failed ticket emails.

6.8 Development Plan and Timeline

The following is a development plan for the Smart Cinema System.

Phase	Tasks	Primary owner
Week 1	Finalize software architecture, UML diagrams, and database design.	All members
Week 2-3	Develop user accounts, movie listings, showtimes, and seat reservations.	Minh Nguyen and Khang Do
Week 4-5	Develop payment processing, refunds, QR tickets, and ticket validation.	Phu Pham and Khang Do
Week 6	Test the complete system, fix errors, and review the final documentation.	All members; cross-review each other's modules

Team Responsibilities

Khang Do: Responsible for seat reservations, booking management, cancellations, and booking tests.

Minh Nguyen: Responsible for user accounts, staff permissions, movie listings, showtimes, and user interface design.

Phu Pham: Responsible for payment processing, refunds, QR ticket generation, ticket validation, and email services.

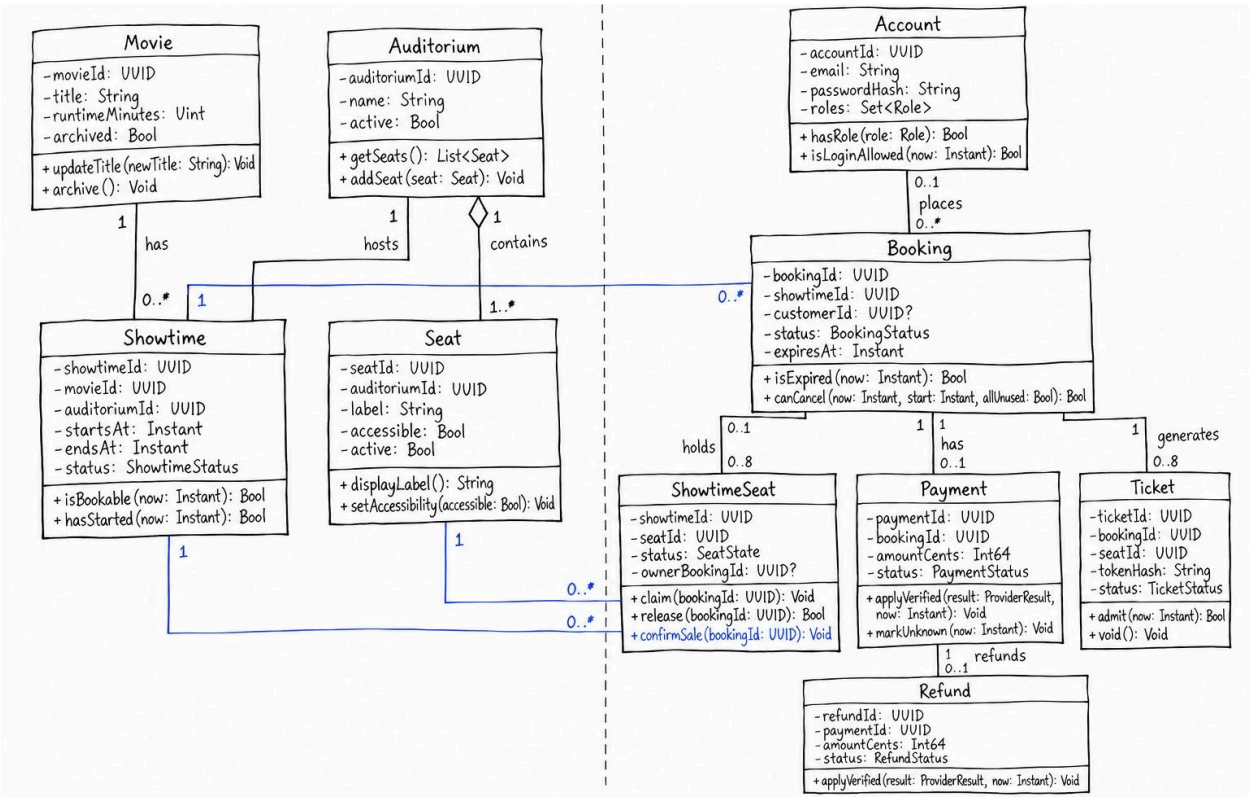
All members will work together to test the system, review each other's work, and update the project documentation.

7. Test Plan

This plan tests the main features of Smart Cinema System: account roles, seat booking, payment, ticket admission, cancellation and refunds. The Excel workbook contains 10 sample test cases in six sets: two unit sets, two integration/functional sets and two system sets.

7.1 Design Specification Update

The existing class design from Assignment 2 is retained because it supports the planned tests. The test targets are listed in Section 7.2.



7.2 Test Approach and Test Sets

Verification is used to determine if the system meets the requirements of the SRS and Design. Unit Testing is used to verify the performance of individual methods. Integration Testing is used to ensure that all related units are passing data correctly to each other. Integration Testing will be used to test the booking and payment features as well, which are part of the functional requirements for this assignment.
Validation is used to assess whether or not both customers and staff will be able to perform their required tasks. System Testing will be conducted by following the entire process of purchasing/admitting onto the system and then cancelling/refunding. During System Testing we will compare what actually happens during each step of the process to what we expect to happen in Excel.

Sample data

Use sample customer, usher and manager accounts. Show S1 starts at 7:00 PM and includes seats A1 and A2. Times use the venue time zone. The sample price is $12 per seat with zero test fees/tax. Reset sample bookings and seats between independent tests.

Set	Level	Target and test data
U1	Unit	Account.hasRole. Manager role: true. Customer without Manager role: false. Finds incorrect role checks. SRS 6.6.
U2	Unit	Booking.canCancel. 7:00 PM show: 6:30 PM is allowed; 6:40 PM is rejected, with unused tickets. Finds cutoff errors. SRS 2.5, 6.6.
I1	Integration / functional	BookingService, Booking and ShowtimeSeat. A1/A2 free: hold both for 5 minutes. A2 already held: reject the selection. Finds partial holds and inconsistent seat ownership. SRS 3.3.3, 6.5-6.7.
I2	Integration / functional	PaymentService, Booking and TicketService. Verified success: confirmed booking and QR ticket. Decline: no confirmation or ticket. Finds payment-to-ticket errors. SRS 3.6, 6.5, 6.7.
S1	System	Purchase and admission. Search, select, pay, receive a ticket and scan twice. Only the first scan admits. Finds failures across the customer and usher flow. SRS 1.2, 2.5, 6.1-6.7.
S2	System	Cancellation and refund. Cancel an unused 7:00 PM booking at 6:00 PM. Refund the full payment, release the seat and reject the ticket. Finds incomplete cancellation changes. SRS 2.5, 6.1, 6.7.

7.3 Test Cases and Execution

Detailed prerequisites, steps and expected results are in Smart Cinema Test Cases.xlsx. This summary uses the same case IDs.

Running and recording tests

Review the SRS and diagram first. Run unit, integration and system tests in that order when the software is available. Use sample objects for unit tests and simulated payment, email and refund responses for other tests. Record the actual result and tester name in Excel. Mark Pass when the expected result is met; otherwise mark Fail and describe the problem. Retest after a fix.

These are planned tests: Actual Result is Not executed, Status is Not Run, and Test Executed By is TBD. P0 means critical booking, payment or ticket behavior; P1 means an important individual

rule. Complete the sample suite by running all ten cases and correcting and retesting any failures.

GitHub repository: https://github.com/khangdo0410/Smart-Cinema-System-Project-SRS

Commit link:
Khang Do:
https://github.com/khangdo0410/Smart-Cinema-System-Project-SRS/commit/05c1b90ab7cb33afd96b3f5a06a7856061cb36de

Minh Nguyen:
https://github.com/khangdo0410/Smart-Cinema-System-Project-SRS/commit/74bffff1c4f1fd28c2884cd236a3667478472b87

Phu Pham:
https://github.com/khangdo0410/Smart-Cinema-System-Project-SRS/commit/75df9f114890ec6ea8006ac22fcf5029556e4c2d